Lecture 3
Basic Neural Networks II

Ying CAO
SIST, ShanghaiTech
Spring, 2026

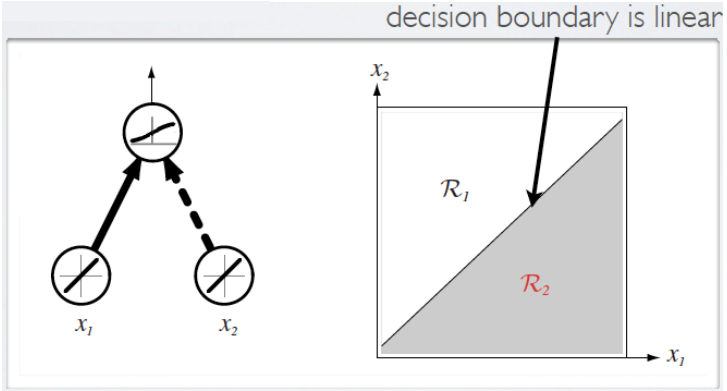
Outline

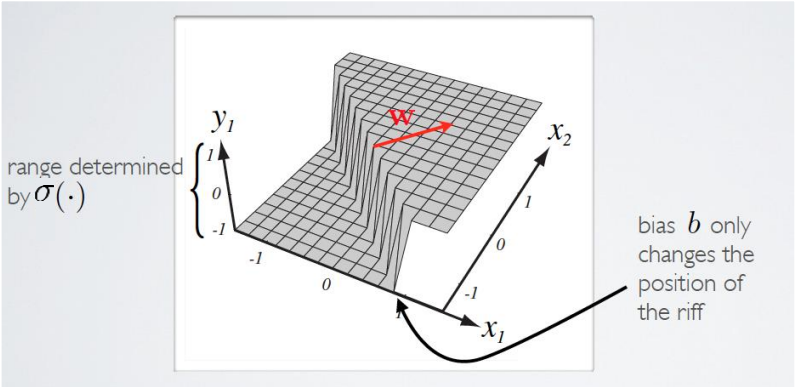
- Multi-layer neural networks
  - Limitations of single-layer networks
  - Networks with single hidden layer
  - Deep networks
- Inference and learning
  - Forward propagation and Backpropagation
  - General BP algorithm

Acknowledgement: Hugo Larochelle's, Mehryar Mohri's and Yingyu Liang's course notes

Capacity of single neuron

- Binary classification
  - A neuron estimates $\sigma(\mathbf{w}^T\mathbf{x})$
  - Its decision boundary is linear, determined by its weights



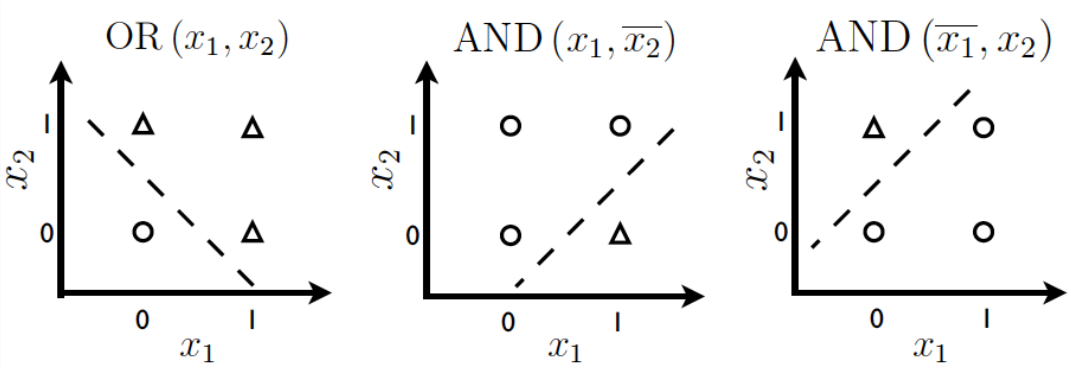


Capacity of single neuron

- Can solve linearly separable problems

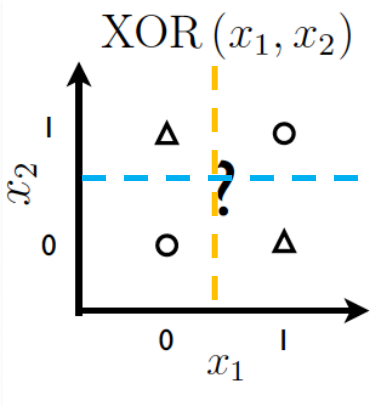
$$\mathcal{D} = \mathcal{D}^+ \cup \mathcal{D}^-$$
$$\exists\ \mathbf{w}^*, \mathbf{w}^{*\mathsf{T}}\mathbf{x} > 0,\ \forall \mathbf{x} \in \mathcal{D}^+$$
$$\mathbf{w}^{*\mathsf{T}}\mathbf{x} < 0,\ \forall \mathbf{x} \in \mathcal{D}^-$$

  - Examples



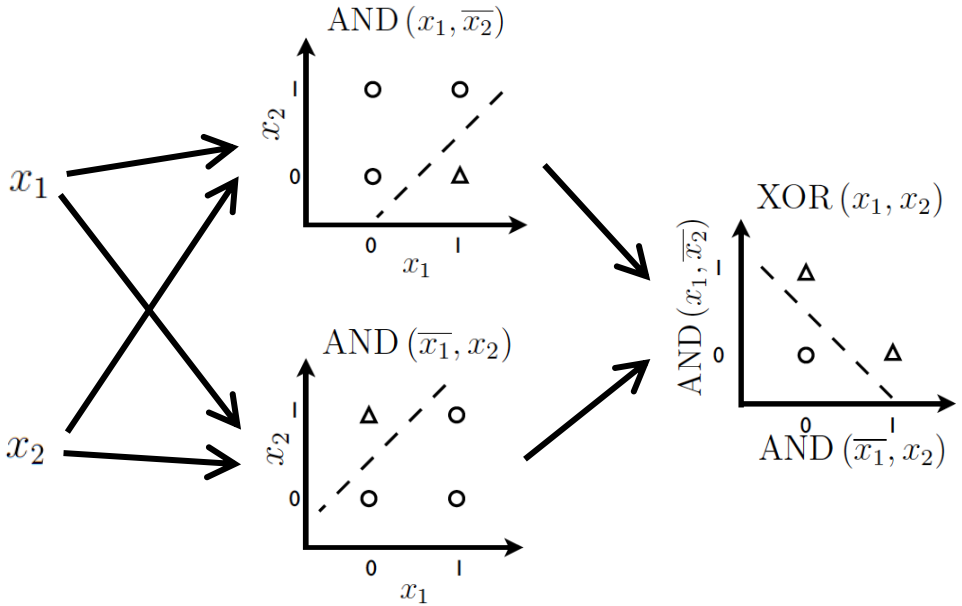
Capacity of single neuron

- Can’t solve linearly non-separable problems



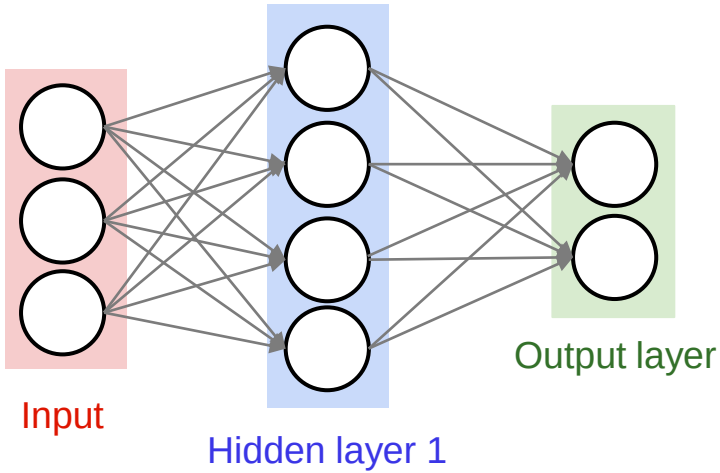
Capacity of single neuron

- Can’t solve linearly non-separable problems
- Add one more layer of neurons
  - Transform the inputs into another space where the problem is linearly separable



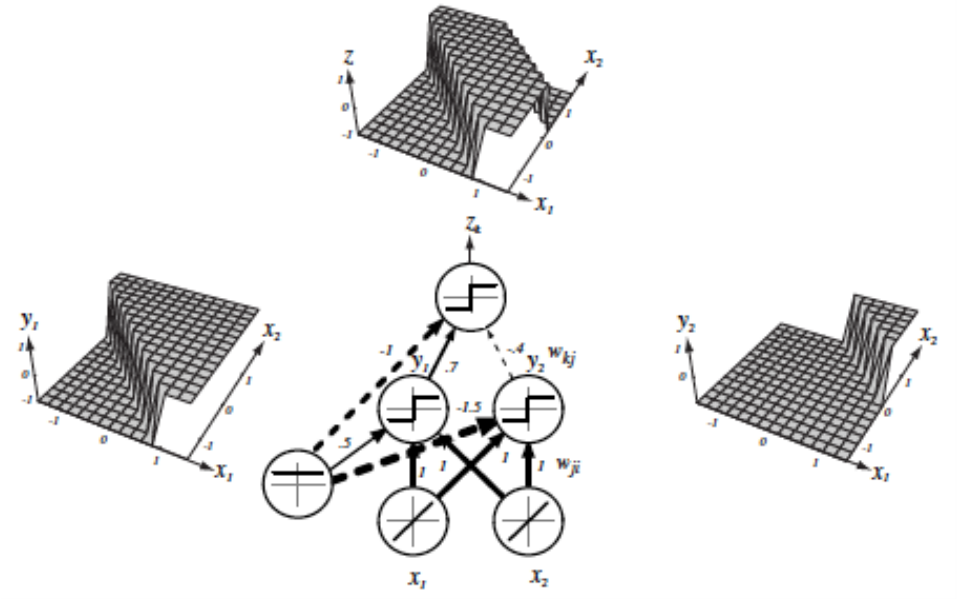
Adding one more layer

- Neural network with single hidden layer
  - 2-layer neural network: ignoring input units



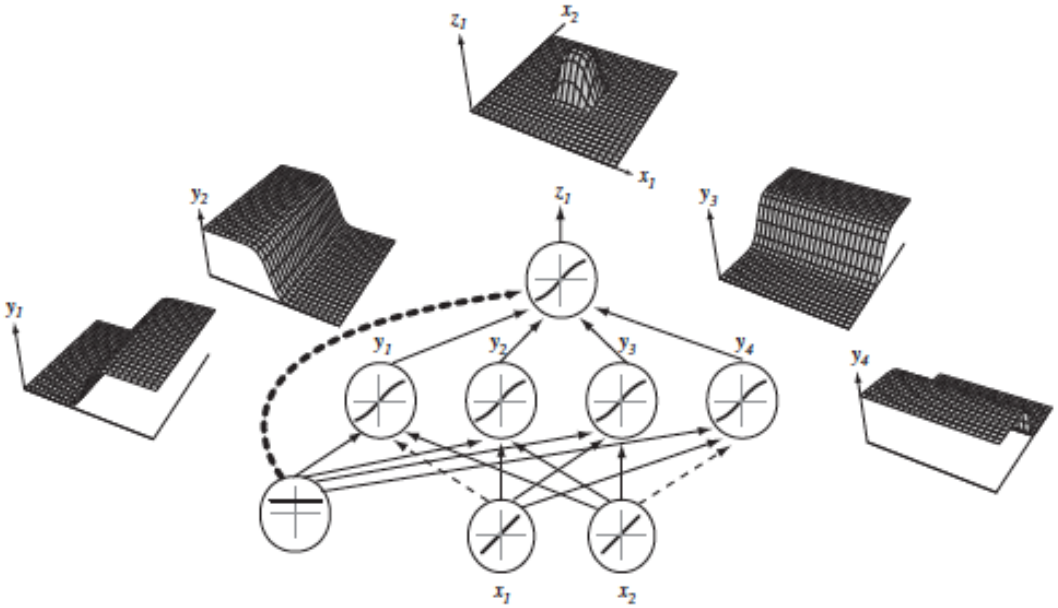
Capacity of neural network

- Neural network with single hidden layer
  - Compose simple functions into a more complex function
  - Partition the input space into more complex regions



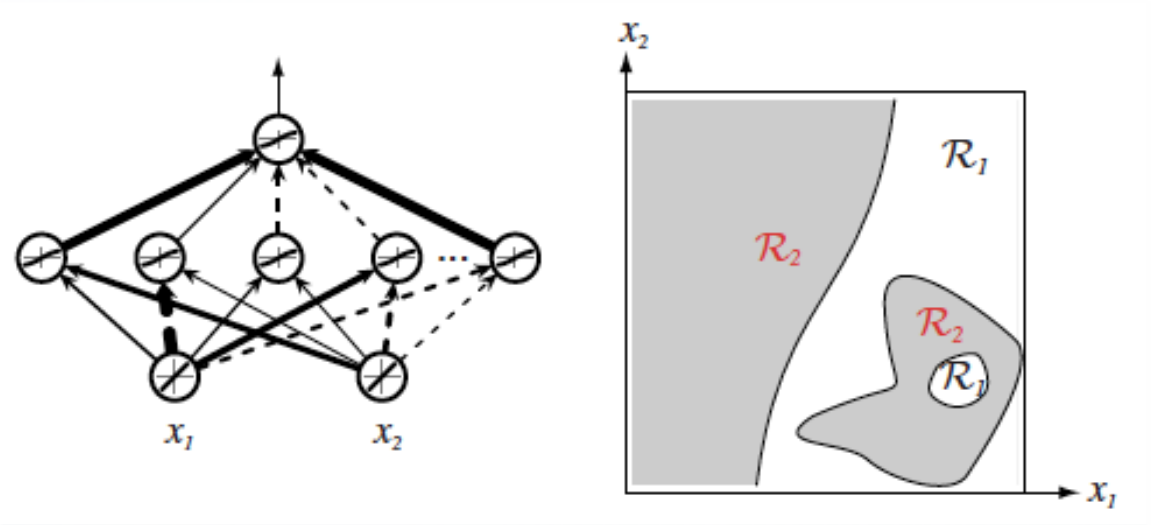
Capacity of neural network

- Neural network with single hidden layer
  - 4 hidden units with sigmoid non-linearity



Capacity of neural network

- Neural network with single hidden layer
  - More complex function (input space partitioning) with more hidden units



Capacity of neural network

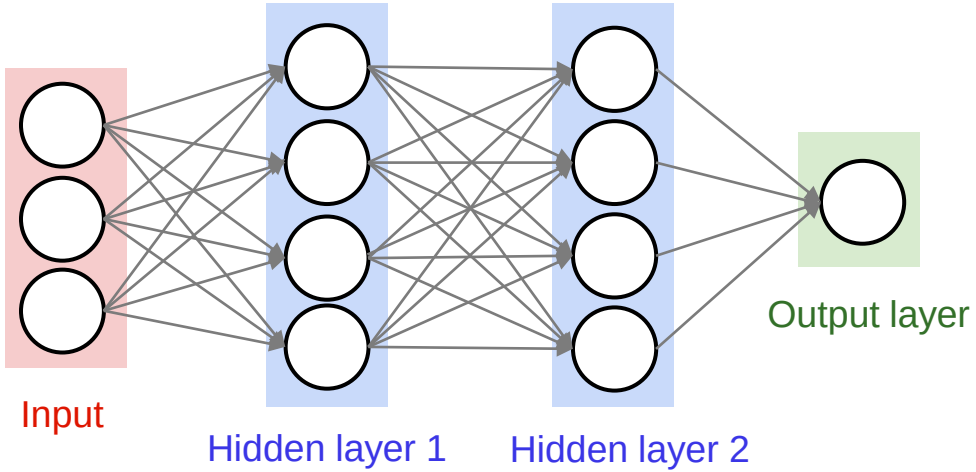
- Universal approximation theorem [Hornik et al., 1989]
  - A single-hidden-layer neural network with a non-linear activation function can approximate any continuous function arbitrarily well, given enough hidden units

- But how in practice?
  - How many hidden units?
  - How to find the parameters by a learning algorithm?

Hornik et al., “Multilayer feedforward networks are universal approximators”, Neural networks, 1989

General neural network

- Multi-layer neural network



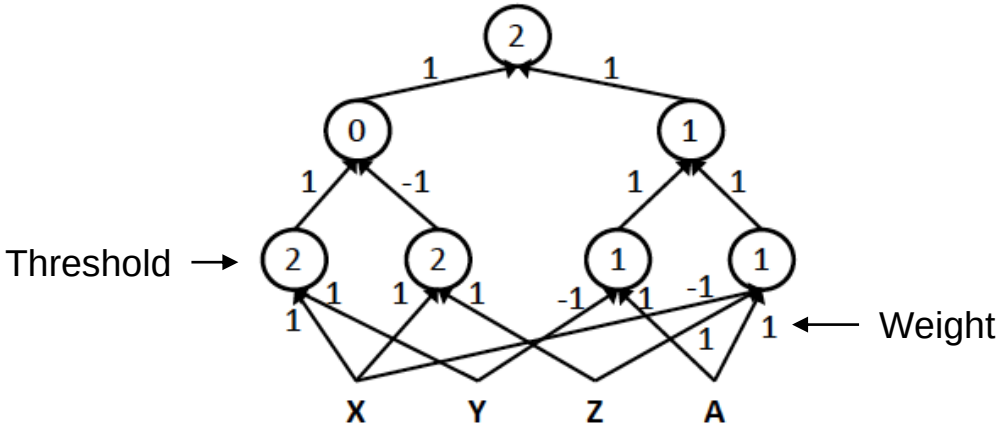
A 3-layer neural net with 3 input units, 4 hidden units in the first and second hidden layers, and 1 output unit

  - Naming convention: N-layer neural net = N-1 layers of hidden units + 1 output layer

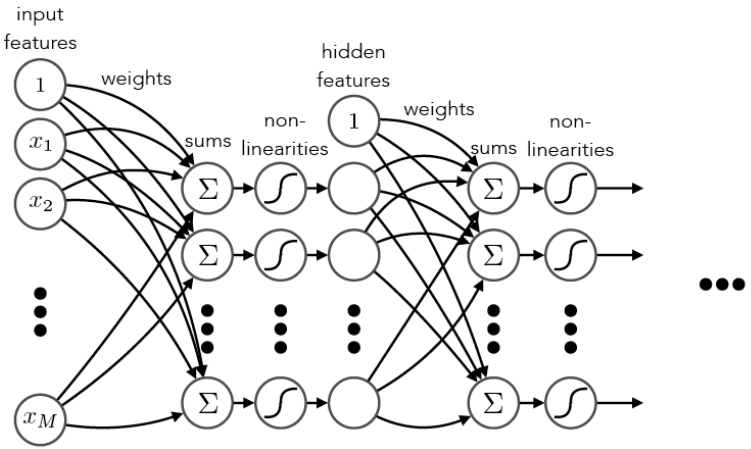
Multi-layer perceptron

- Boolean case
  - Multi-layer perceptrons (MLPs) can compute more complex Boolean functions
  - MLPs are universal Boolean functions: MLPs can compute any Boolean function

$$((A\&\bar{X}\&Z)|(A\&\bar{Y}))\&((X\ \&\ Y)|\overline{(X\&Z)})$$



Multi-layer neural network



1st layer

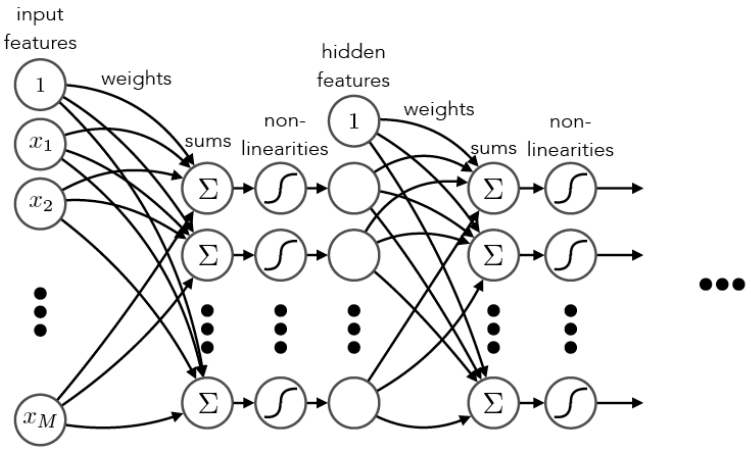
$$\mathbf{s}^{(1)} = \mathbf{W}^{(1)\intercal}\mathbf{x}^{(0)}$$
$$\mathbf{x}^{(1)} = \sigma(\mathbf{s}^{(1)})$$

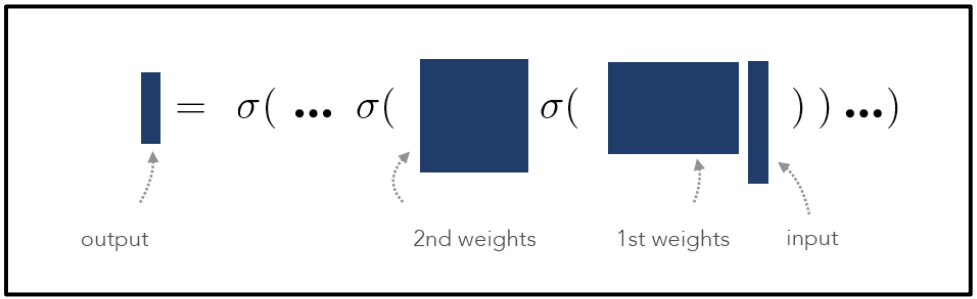
2nd layer

$$\mathbf{s}^{(2)} = \mathbf{W}^{(2)\intercal}\mathbf{x}^{(1)}$$
$$\mathbf{x}^{(2)} = \sigma(\mathbf{s}^{(2)})$$

...

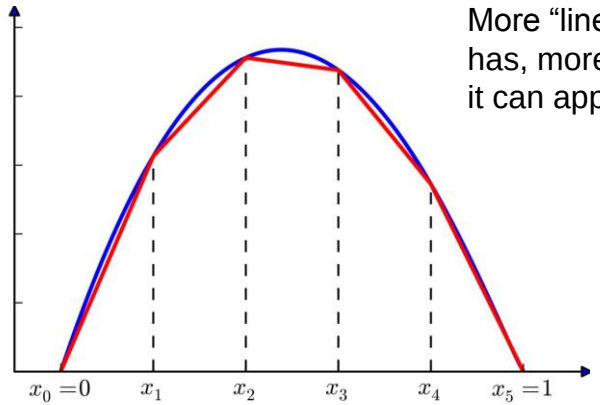
Multi-layer neural network



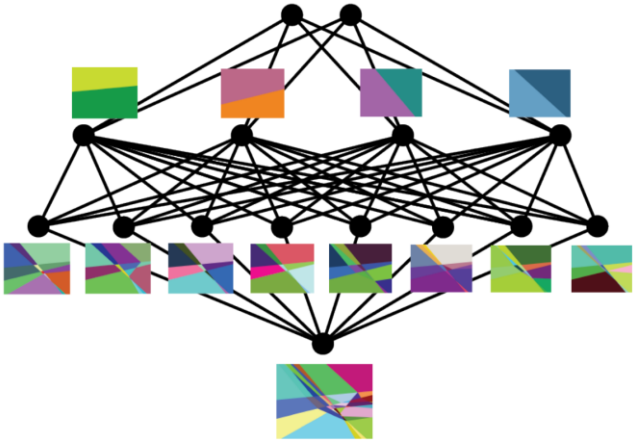


Why more layers (deeper)?

- Deeper networks represent functions more compactly
  - Number of “linear regions”: a measure of model flexibility/complexity



More “linear regions” a model has, more complex functions it can approximate

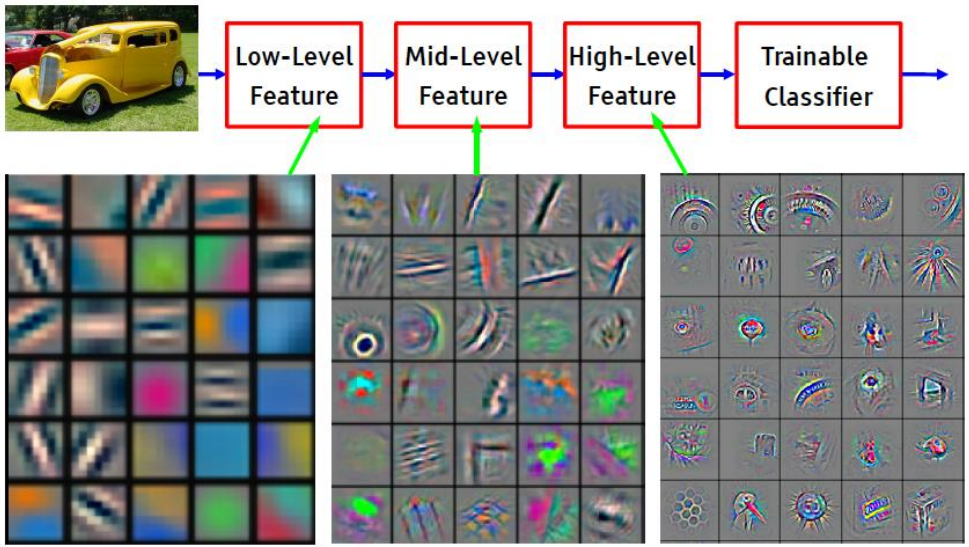


  - Given the same number of hidden units, deeper networks have significantly more “linear regions” than shallower networks [Montufar et al., 2014]
    - 2 layers of 10 hidden units vs. 1 layer of 20 hidden units

Montufar et al., “On the Number of Linear Regions of Deep Neural Networks”, NIPS 2014
Hanin and Rolnick, “Complexity of Linear Regions in Deep Networks”, ICML 2019

Why more layers (deeper)?

- Deeper networks learn hierarchical feature representation
  - Efficient learning: more complex features are composed from simpler features
  - Better generalizability: low-level features are generic



Feature visualization of convolutional net trained on ImageNet from [Zeiler & Fergus, 2013]

Outline

- Multi-layer neural networks
  - Limitations of single-layer networks
  - Networks with single hidden layer
  - Deep networks
- Inference and learning
  - Forward propagation and Backpropagation
  - General BP algorithm

Acknowledgement: Hugo Larochelle's, Mehryar Mohri's and Yingyu Liang's course notes

Computation in neural network

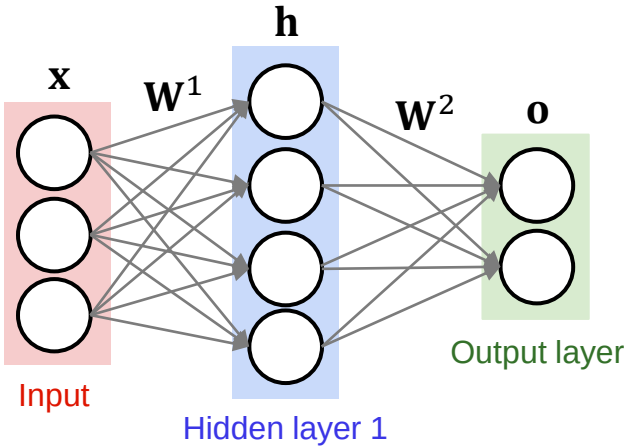
- We only need to know two algorithms
  - Inference/prediction: simply forward pass
  - Parameter learning: backward pass is needed
- Basic facts:
  - A neural network is a function of composed operations f's

$$\mathbf{o} = f_L\left(\mathbf{W}^{(L)}, f_{L-1}\left(\mathbf{W}^{(L-1)}, \ldots, f_1\left(\mathbf{W}^{(1)}, \mathbf{x}\right)\right)\right)$$

  - All f's are differentiable operators that consist of linear and nonlinear transformations

Forward pass

- What does the network compute?



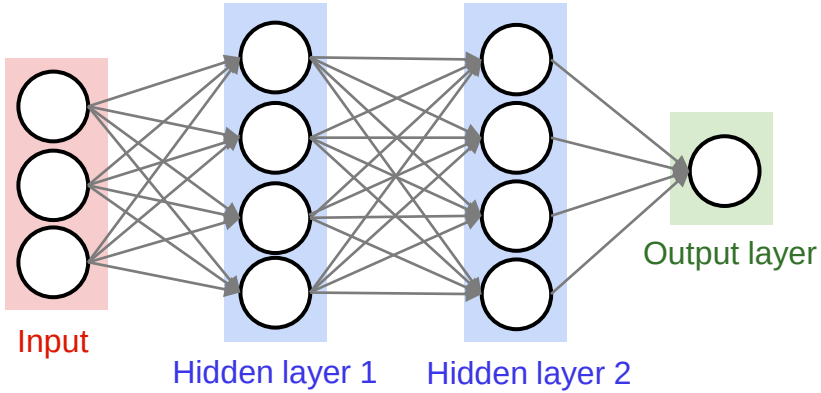
  - Output of the network:

$$h_j = f(\mathbf{W}^1_{j0} + \sum_{i=1}^{D} \mathbf{W}^1_{ji}\, x_i) \qquad \text{(j indexes over hidden units)}$$

$$o_k = g(\mathbf{W}^2_{k0} + \sum_{j=1}^{J} \mathbf{W}^2_{kj}\, h_j) \qquad \text{(k indexes over output units)}$$

Forward pass in Python

- What does the network compute?
  - Example code for forward pass for a 3-layer network in Python




```
# forward-pass of a 3-layer neural network:
f = lambda x: 1.0/(1.0 + np.exp(-x)) # activation function (use sigmoid)
x = np.random.randn(3, 1) # random input vector of three numbers (3x1)
h1 = f(np.dot(W1, x) + b1) # calculate first hidden layer activations (4x1)
h2 = f(np.dot(W2, h1) + b2) # calculate second hidden layer activations (4x1)
out = np.dot(W3, h2) + b3 # output neuron (1x1)
```

  - Can be implemented efficiently using matrix operations

Parameter learning

- Supervised learning framework
  - Find weights:

$$\mathbf{W}^* = \arg\min_{\mathbf{W}} L = \arg\min_{\mathbf{W}} \sum_{n=1}^{N} \text{loss}(\mathbf{o}^{(n)}, \mathbf{t}^{(n)})$$

    where $\mathbf{o} = f(\mathbf{x}; \mathbf{W})$ is the network output, and $\mathbf{t}$ is the target

  - Define a loss function, e.g.,:
    - L2 loss: $\sum_k \left(o_k^{(n)} - t_k^{(n)}\right)^2$
    - Cross entropy loss: $-\sum_k t^{(n)} \log o_k^{(n)}$
  - Gradient descent: $\mathbf{W}_{t+1} = \mathbf{W}_t - \eta \frac{\partial L}{\partial \mathbf{W}_t}$

Backward pass

- Backpropagation
  - An efficient method to calculate the gradients of the loss L w.r.t. network weights $\mathbf{W}$ using chain rule
    - Chain rule: for any nested function $y = f(g(x))$

$$\frac{\partial y}{\partial x} = \frac{\partial y}{\partial g(x)} \frac{\partial g(x)}{\partial x}$$

    - Recall that a neural network is a nested function:

$$\mathbf{o} = f_L\left(\mathbf{W}^{(L)}, f_{L-1}\left(\mathbf{W}^{(L-1)}, \dots, f_1\left(\mathbf{W}^{(1)}, \mathbf{x}\right)\right)\right)$$

$$L = \text{loss}(\mathbf{o}, \mathbf{t})$$

  - Compute gradients of one layer at a time, iterating backward from the last layer

Gradient descent iteration

- Forward pass

1st layer

$$\mathbf{s}^{(1)} = \mathbf{W}^{(1)\intercal}\mathbf{x}^{(0)}$$
$$\mathbf{x}^{(1)} = \sigma(\mathbf{s}^{(1)})$$

2nd layer

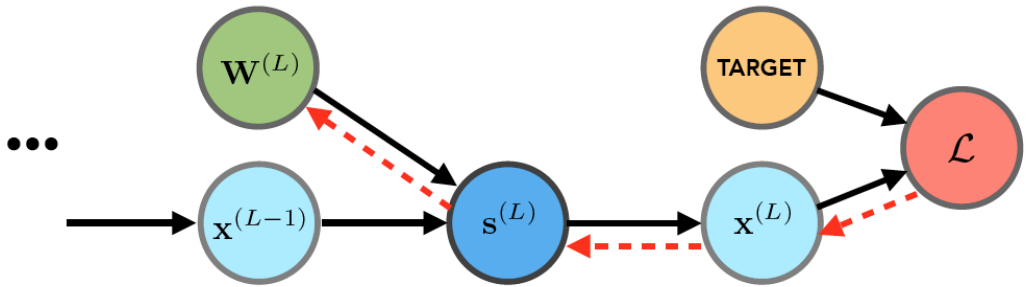
$$\mathbf{s}^{(2)} = \mathbf{W}^{(2)\intercal}\mathbf{x}^{(1)}$$
$$\mathbf{x}^{(2)} = \sigma(\mathbf{s}^{(2)})$$

$\bullet\bullet\bullet$

Loss

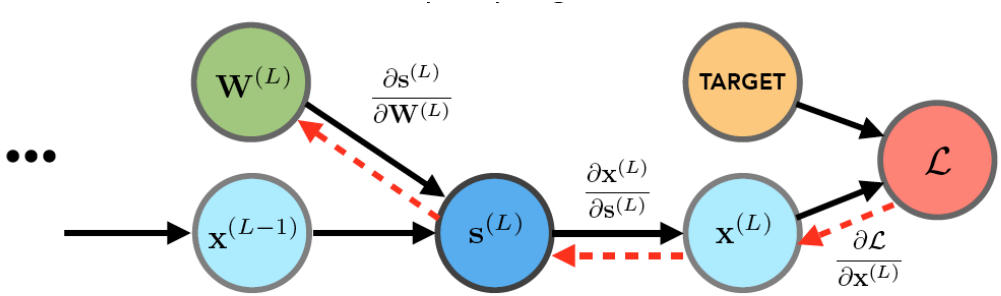
$$\mathcal{L}$$

- Backward pass
  - Calculate $\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(1)}}, \frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(2)}}$, ...; start with the final layer: $\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(L)}}$
  - To determine the chain rule ordering, draw the dependency graph



Gradient descent iteration

- Backward pass

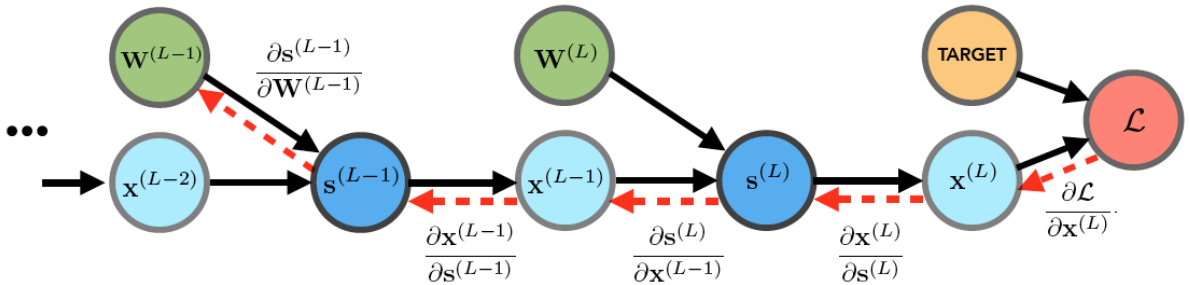


$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(L)}} = \underbrace{\frac{\partial \mathcal{L}}{\partial \mathbf{x}^{(L)}} \frac{\partial \mathbf{x}^{(L)}}{\partial \mathbf{s}^{(L)}}}_{\frac{\partial \mathcal{L}}{\partial \mathbf{s}^{(L)}}} \frac{\partial \mathbf{s}^{(L)}}{\partial \mathbf{W}^{(L)}}$$

$\frac{\partial \mathcal{L}}{\partial \mathbf{s}^{(L)}}$ can be computed and stored

Gradient descent iteration

- Backward pass
  - Go back one more layer



$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}^{(L-1)}} = \underbrace{\frac{\partial \mathcal{L}}{\partial \mathbf{x}^{(L)}} \frac{\partial \mathbf{x}^{(L)}}{\partial \mathbf{s}^{(L)}}} \frac{\partial \mathbf{s}^{(L)}}{\partial \mathbf{x}^{(L-1)}} \frac{\partial \mathbf{x}^{(L-1)}}{\partial \mathbf{s}^{(L-1)}} \frac{\partial \mathbf{s}^{(L-1)}}{\partial \mathbf{W}^{(L-1)}}$$

Past gradient $\frac{\partial \mathcal{L}}{\partial \mathbf{s}^{(L)}}$ can be reused

Outline

- Multi-layer neural networks
  - Limitations of single-layer networks
  - Networks with single hidden layer
  - Deep networks
- Inference and learning
  - Forward propagation and Backpropagation
  - General BP algorithm

Acknowledgement: Hugo Larochelle's, Mehryar Mohri's and Yingyu Liang's course notes

Example

- Univariate logistic least square model
  - Regression using a single neuron with sigmoid non-linearity and a single input

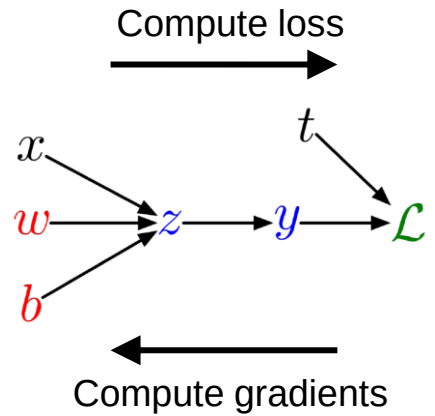
$$z = wx + b$$
$$y = \sigma(z)$$
$$\mathcal{L} = \frac{1}{2}(y - t)^2$$

  - How to implement backpropagation on this model?

Computation graph

- Represent the computations using a computation graph
  - Nodes: inputs & parameters & computed quantities
  - Edges: a node is computed directly as a function of which other nodes

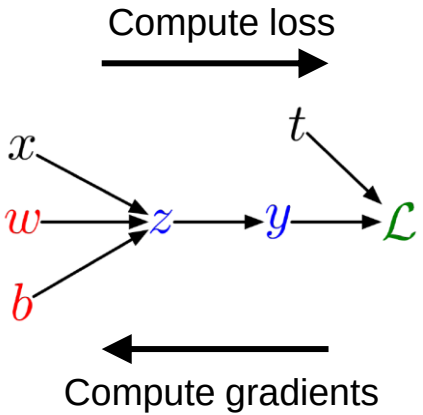
$$z = wx + b$$
$$y = \sigma(z)$$
$$\mathcal{L} = \frac{1}{2}(y - t)^2$$



- Compute loss (forward) and gradients (backward) on the graph

Univariate chain rule

- The computation graph has only fan-out = 1



Compute loss:

$$z = wx + b$$
$$y = \sigma(z)$$
$$\mathcal{L} = \frac{1}{2}(y - t)^2$$

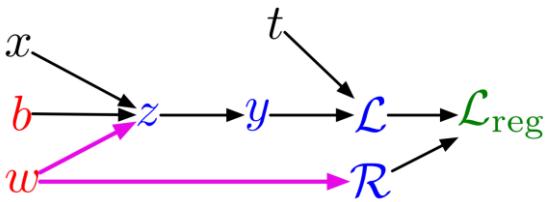
Compute gradients:

$$\frac{\partial \mathcal{L}}{\partial y} = y - t$$
$$\frac{\partial \mathcal{L}}{\partial z} = \frac{\partial \mathcal{L}}{\partial y}\sigma'(z)$$
$$\frac{\partial \mathcal{L}}{\partial w} = \frac{\partial \mathcal{L}}{\partial z}x$$
$$\frac{\partial \mathcal{L}}{\partial b} = \frac{\partial \mathcal{L}}{\partial z}$$

Multivariate chain rule

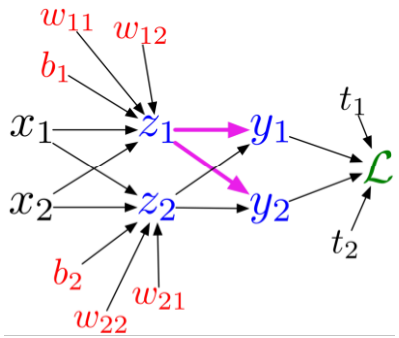
- The computation graph has fan-out > 1

L_2-regularized regression



$$z = wx + b$$
$$y = \sigma(z)$$
$$\mathcal{L} = \frac{1}{2}(y - t)^2$$
$$\mathcal{R} = \frac{1}{2}w^2$$
$$\mathcal{L}_{\text{reg}} = \mathcal{L} + \lambda\mathcal{R}$$

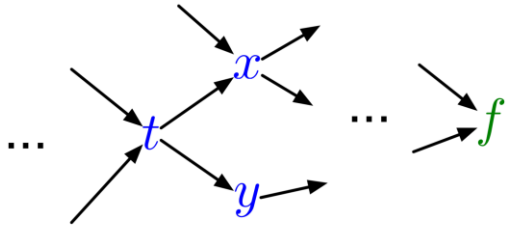
Multiclass logistic regression



$$z_l = \sum_j w_{lj}x_j + b_l$$
$$y_k = \frac{e^{z_k}}{\sum_l e^{z_l}}$$
$$\mathcal{L} = -\sum_k t_k \log y_k$$

Multivariate chain rule

- A computation graph
  - There are directed edges from t to x and y (both x and y depends on t): t is the parent of x and y
  - x and y are the children of t



$$\frac{\partial f}{\partial t} = \frac{\partial f}{\partial x}\frac{\partial x}{\partial t} + \frac{\partial f}{\partial y}\frac{\partial y}{\partial t}$$

General backpropagation

- Suppose we have a computation graph:
  - Let $v_1, \ldots, v_N$ be a topological ordering of the computation graph (i.e., a node is visited only after all the nodes that it depends on are visited)
    - Parents come before children
  - v_N denotes the final node (e.g., loss)

Forward pass

For $i = 1, \ldots, N$

Compute v_i as a function of $\mathrm{Pa}(v_i)$

Backward pass

$$\frac{\partial v_N}{\partial v_N} = 1$$

For $i = N-1, \ldots, 1$

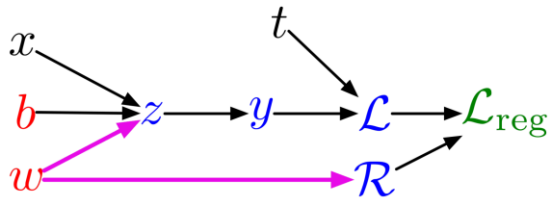
$$\frac{\partial v_N}{\partial v_i} = \sum_{j \in \mathrm{Ch}(v_i)} \frac{\partial v_N}{\partial v_j} \frac{\partial v_j}{\partial v_i}$$

$\mathrm{Pa}(v_i)$: all parents of v_i

$\mathrm{Ch}(v_i)$: all children of v_i

General backpropagation

- L_2-regularized regression



Forward pass:

$$
\begin{aligned}
z &= wx + b \\
y &= \sigma(z) \\
\mathcal{L} &= \frac{1}{2}(y - t)^2 \\
\mathcal{R} &= \frac{1}{2}w^2 \\
\mathcal{L}_{\text{reg}} &= \mathcal{L} + \lambda\mathcal{R}
\end{aligned}
$$

Backward pass:

$$
\begin{aligned}
\frac{\partial \mathcal{L}_{\text{reg}}}{\partial \mathcal{L}_{\text{reg}}} &= 1 \\
\frac{\partial \mathcal{L}_{\text{reg}}}{\partial \mathcal{R}} &= \frac{\partial \mathcal{L}_{\text{reg}}}{\partial \mathcal{L}_{\text{reg}}}\frac{\partial \mathcal{L}_{\text{reg}}}{\partial \mathcal{R}} \\
&= \lambda \\
\frac{\partial \mathcal{L}_{\text{reg}}}{\partial \mathcal{L}} &= \frac{\partial \mathcal{L}_{\text{reg}}}{\partial \mathcal{L}_{\text{reg}}}\frac{\partial \mathcal{L}_{\text{reg}}}{\partial \mathcal{L}} \\
&= 1 \\
\frac{\partial \mathcal{L}_{\text{reg}}}{\partial y} &= \frac{\partial \mathcal{L}_{\text{reg}}}{\partial \mathcal{L}}\frac{\partial \mathcal{L}}{\partial y} \\
&= (y - t)
\end{aligned}
$$

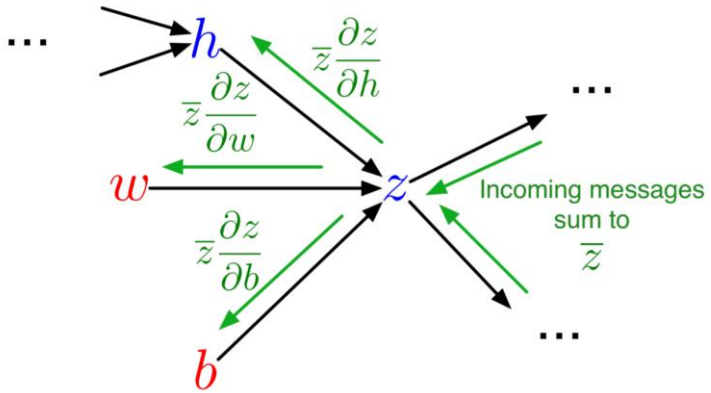
$$
\begin{aligned}
\frac{\partial \mathcal{L}_{\text{reg}}}{\partial z} &= \frac{\partial \mathcal{L}_{\text{reg}}}{\partial y}\frac{\partial y}{\partial z} \\
&= (y - t)\sigma'(z)
\end{aligned}
$$

$$
\begin{aligned}
\frac{\partial \mathcal{L}_{\text{reg}}}{\partial w} &= \frac{\partial \mathcal{L}_{\text{reg}}}{\partial z}\frac{\partial z}{\partial w} + \frac{\partial \mathcal{L}_{\text{reg}}}{\partial \mathcal{R}}\frac{\partial \mathcal{R}}{\partial w} \\
&= (y - t)\sigma'(z)x + \lambda w
\end{aligned}
$$

$$
\begin{aligned}
\frac{\partial \mathcal{L}_{\text{reg}}}{\partial b} &= \frac{\partial \mathcal{L}_{\text{reg}}}{\partial z}\frac{\partial z}{\partial b} \\
&= (y - t)\sigma'(z)
\end{aligned}
$$

General backpropagation

- Backprop as message passing:

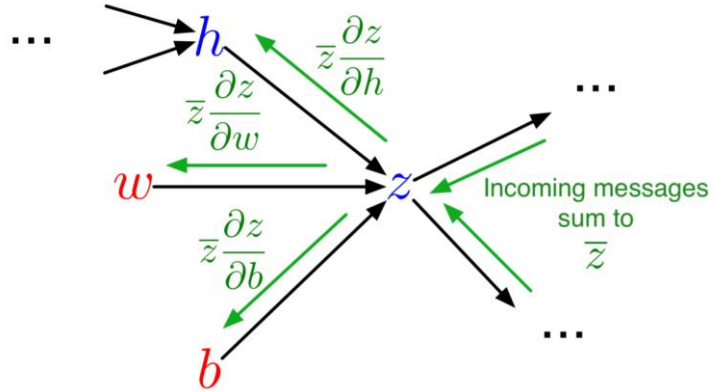


$\bar{z} = \frac{\partial L}{\partial z}$ (L is the final node)

  - Each node receives a set of messages from its children, which are aggregated into the gradient of the node; then the node passes messages to its parents
  - Local computation on the graph: each node only needs to know how to compute gradients w.r.t. its inputs

Computation cost

- Forward pass: one add-multiply operation per weight
- Backward pass: two add-multiply operations per weight



Forward pass: $z_k = \sum_i w_{ki} h_i + b_k$

Backward pass: $\overline{w_{ki}} = \overline{z}_k h_i$

$$\overline{h_i} = \sum_k \overline{z}_k w_{ki}$$

- The cost is linear in the number of layers k, quadratic in the number of units per layer n
  - Number of weights $\approx kn^2$

Summary

- Multi-layer neural networks
- Inference and learning
  - Forward propagation and Backpropagation

- Next time
  - More on backpropagation
  - Convolutional Neural Network (CNN) Basics